ECS Project Step 2.2 – Testing Docker Image

To Begin with the Lab

Summary of the Lab

In this lab, the Docker image created in the previous step was run locally on a Docker-enabled **Amazon EC2** instance to verify that the PHP application was working before deployment to **Amazon ECS**. The container was started with runtime environment variables so the **Amazon S3** bucket name and AWS Region could be passed dynamically instead of being hardcoded in the source code. The application was then tested by uploading an image, and the uploaded file was confirmed in the configured **Amazon S3** bucket. The video also explained why AWS Access Keys should not be stored inside application code and introduced **IAM Roles** as the recommended best practice for secure AWS access.

Understanding Environment Variables and IAM Roles

Environment variables let an application receive values like a bucket name or Region when the container starts, instead of storing those values inside the code. This exists so the same Docker image can be reused in different environments without rebuilding it every time a setting changes. **IAM Roles** provide temporary AWS permissions to a running application without exposing long-term Access Keys or Secret Keys in the source code. For example, a PHP app that uploads images to **Amazon S3** can use an environment variable for the bucket name and an **IAM Role** for secure access, which keeps the deployment flexible and safer.

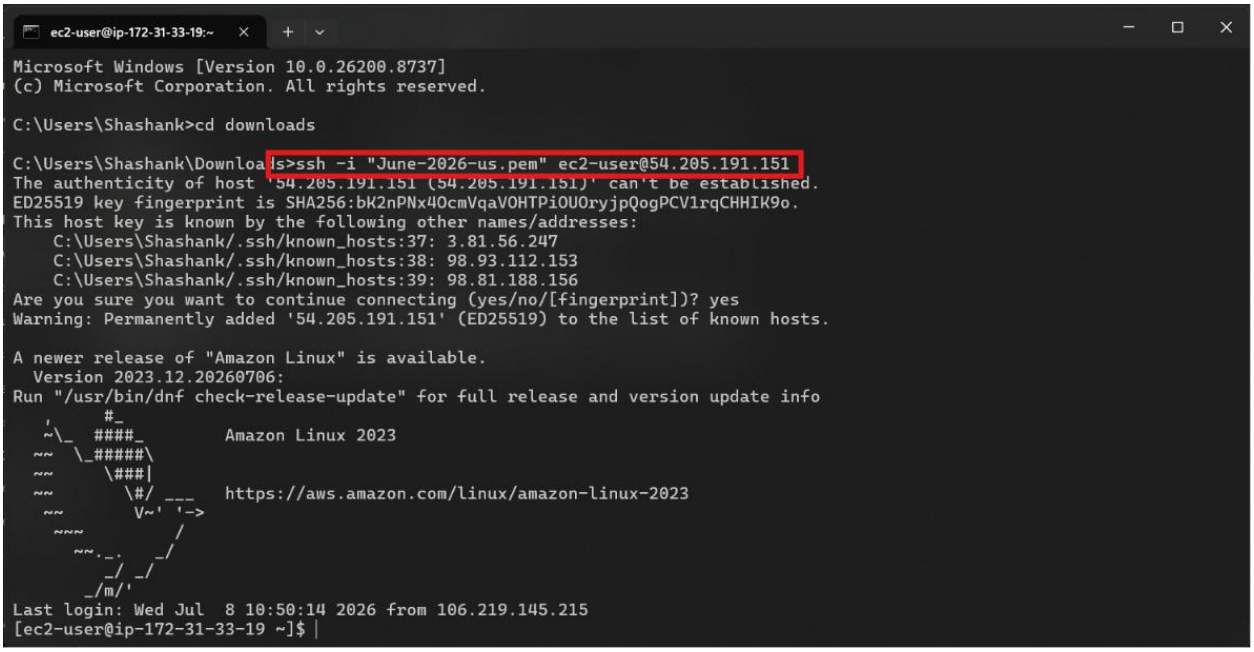
Example:

A PHP application uploads customer profile images to a bucket named cloudfolks-lab-bucket in the ap-south-1 Region. Instead of editing upload.php each time the bucket or Region changes, the values are passed into the container at runtime. Later, when the app runs on **Amazon ECS**, the Task uses an **IAM Role** so the application can upload to **Amazon S3** without hardcoded credentials.

Lab Steps

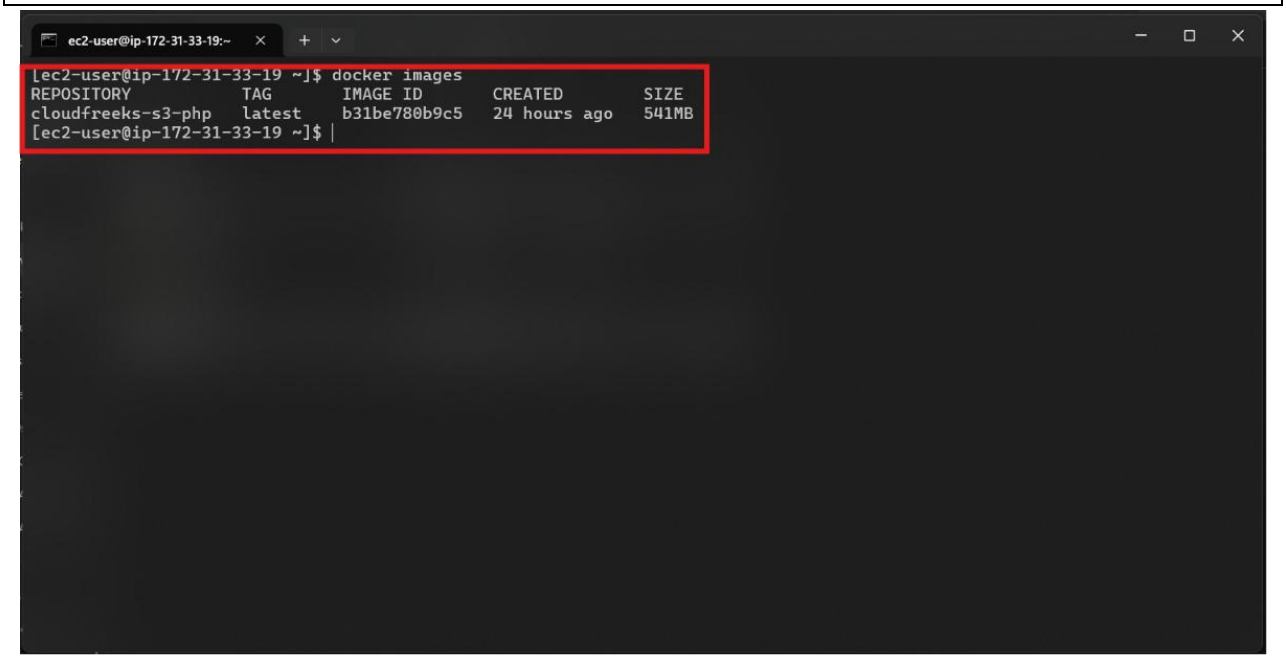
Step 1 – Verify the Docker Image

- Sign in to the **AWS Management Console**.
- Connect to the Docker-enabled **Amazon EC2** build machine using SSH.



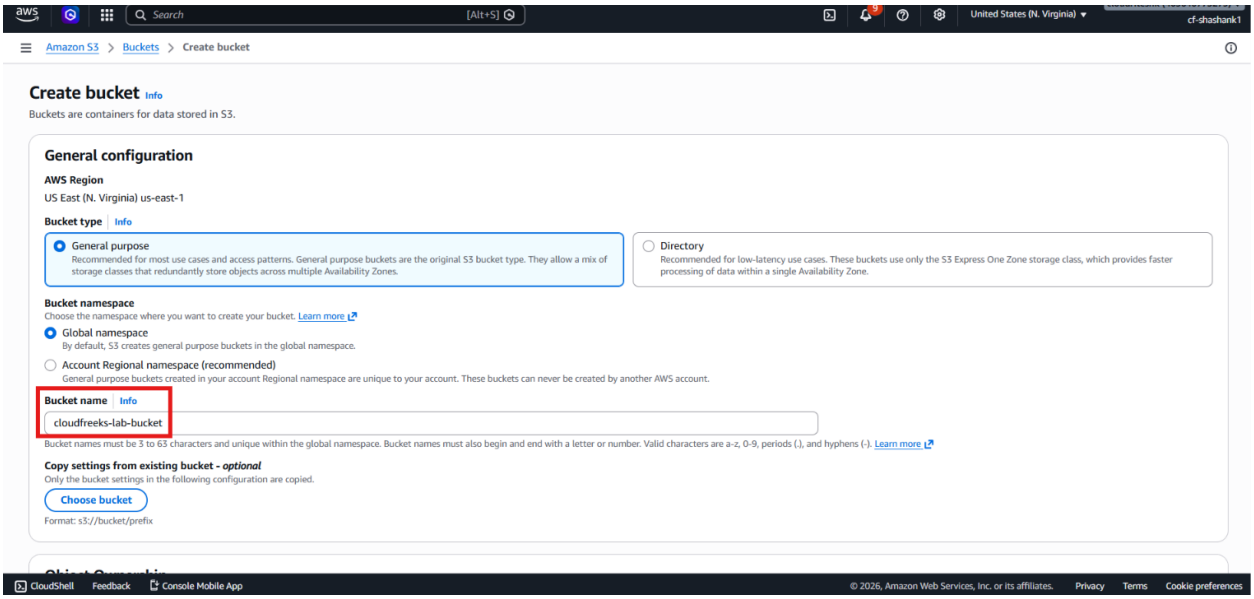
- Verify that the Docker image from the previous lab exists.

```
docker images
```

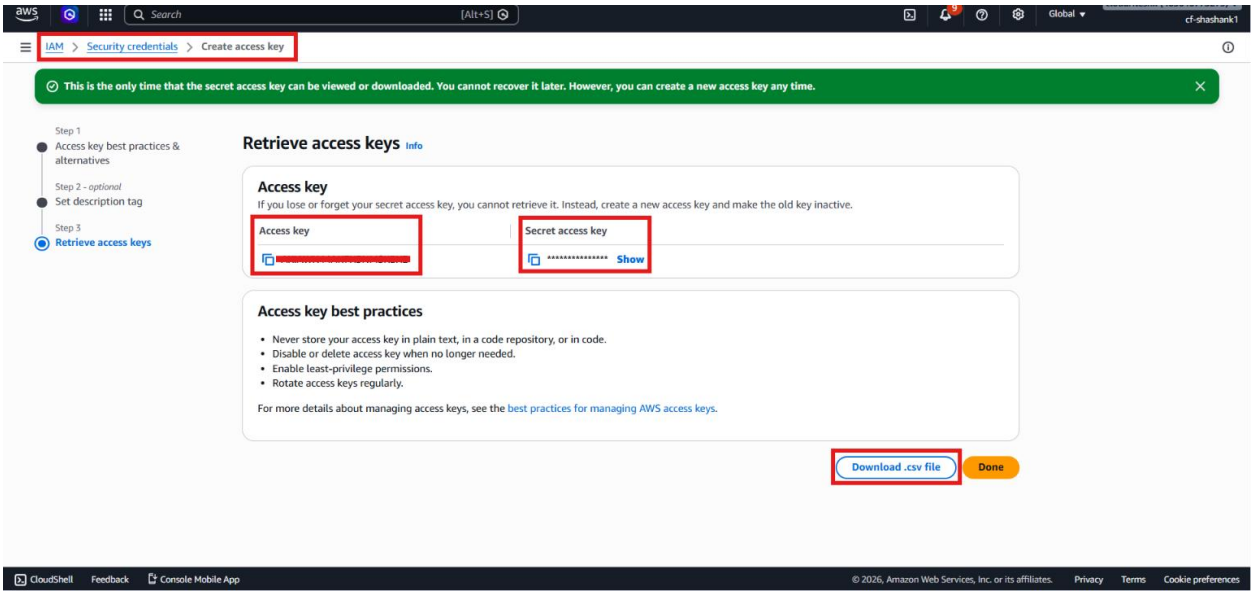


Step 2 – Run the Docker Container

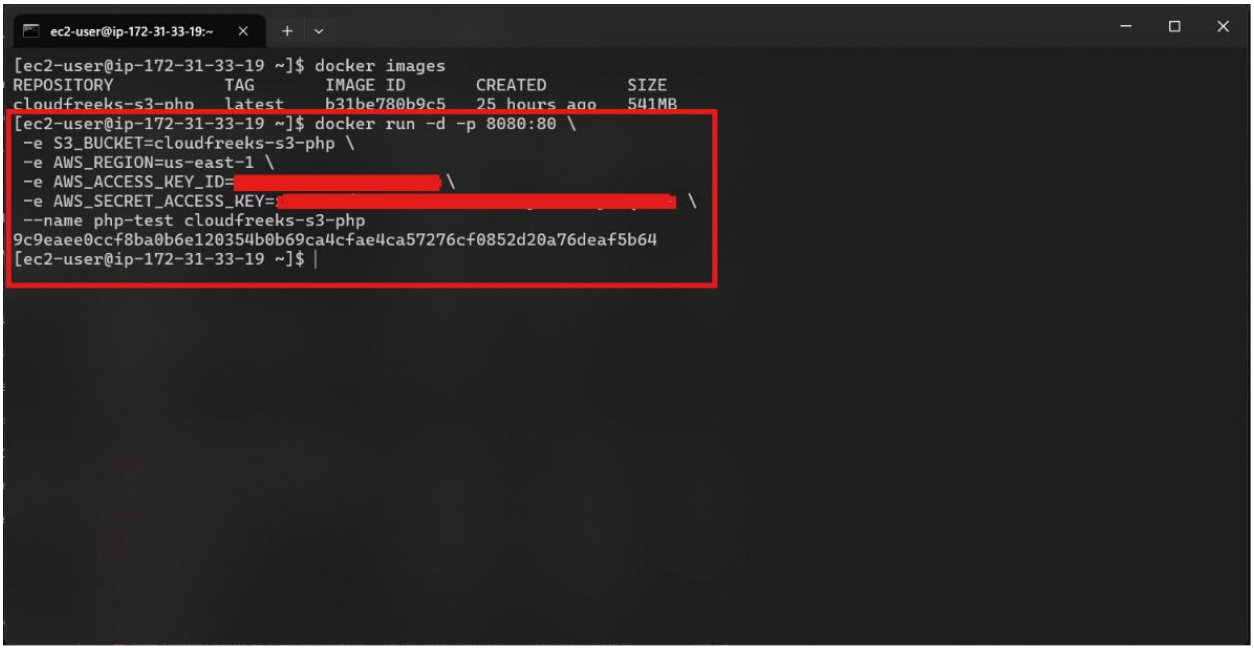
- The commands can be found in the Resources folder
- Run the Docker image in detached mode.
- Map port **8080** on the EC2 instance to port **80** inside the container.
- Create a sample bucket if you want to so I will make one to pass it as the bucket variable



- Pass the **Amazon S3** bucket name as an environment variable.
- Pass the AWS Region as an environment variable.
- Generate an Access Key if you have not made it yet.



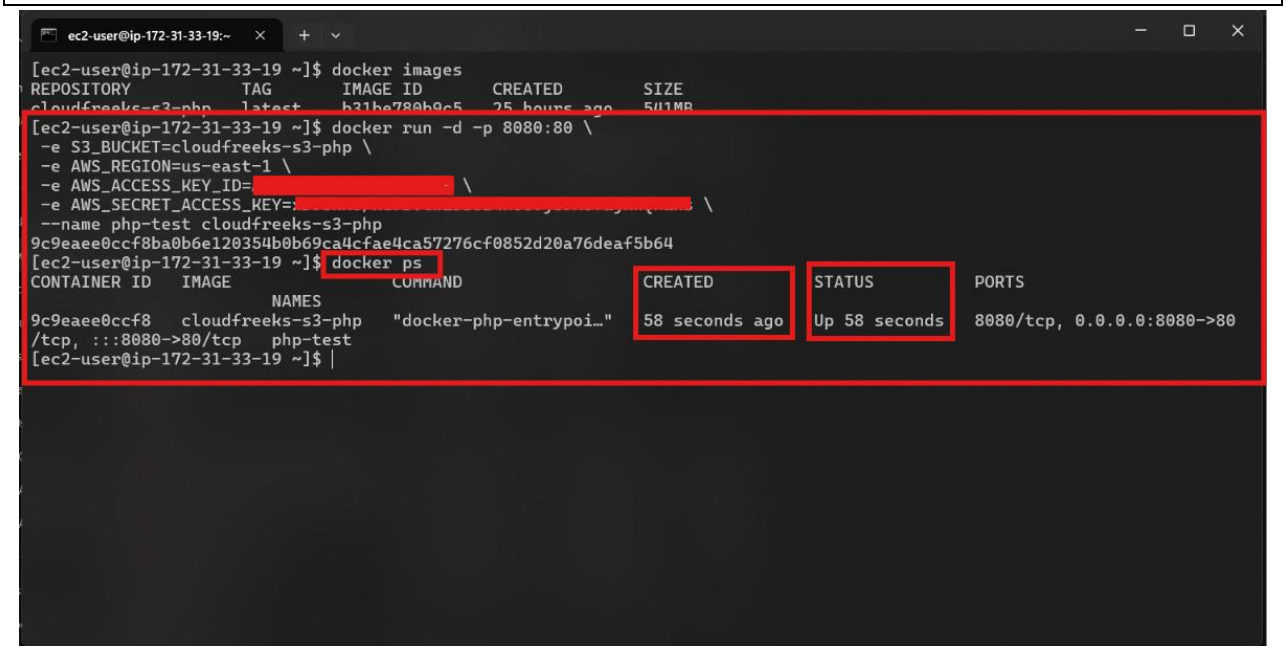
- Pass the AWS Access Key and Secret Access Key temporarily for local testing.



Step 3 – Verify the Running Container

- Verify that the container is running.

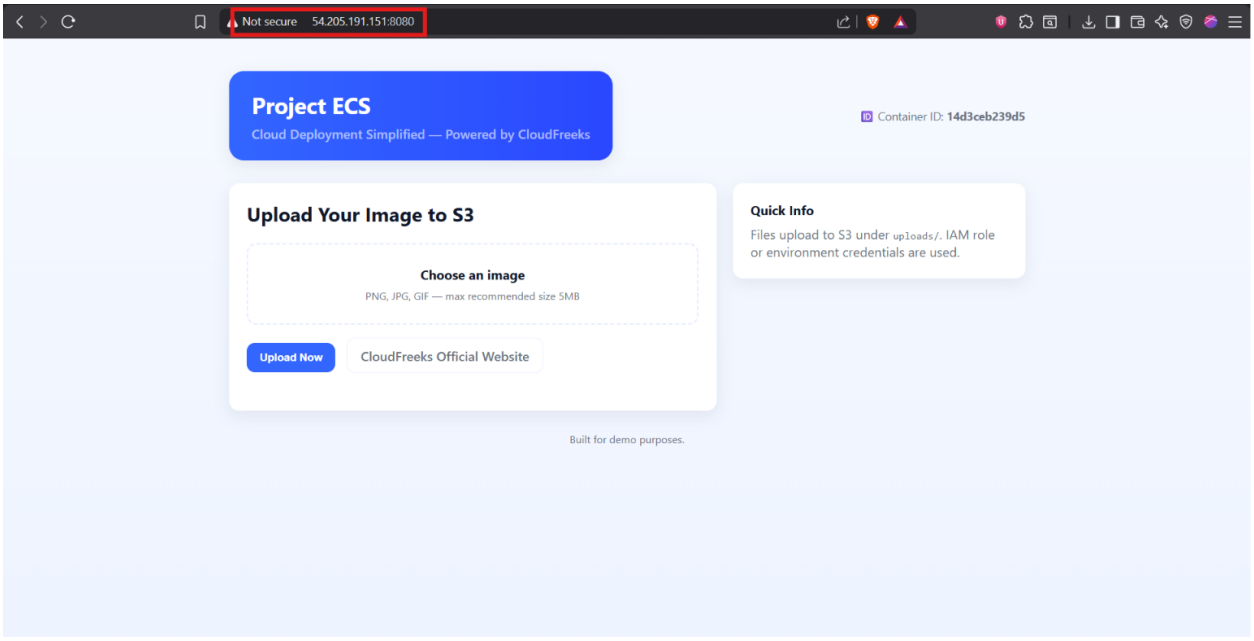
```
docker ps
```



- Open the application using the EC2 public IP on port **8080**.

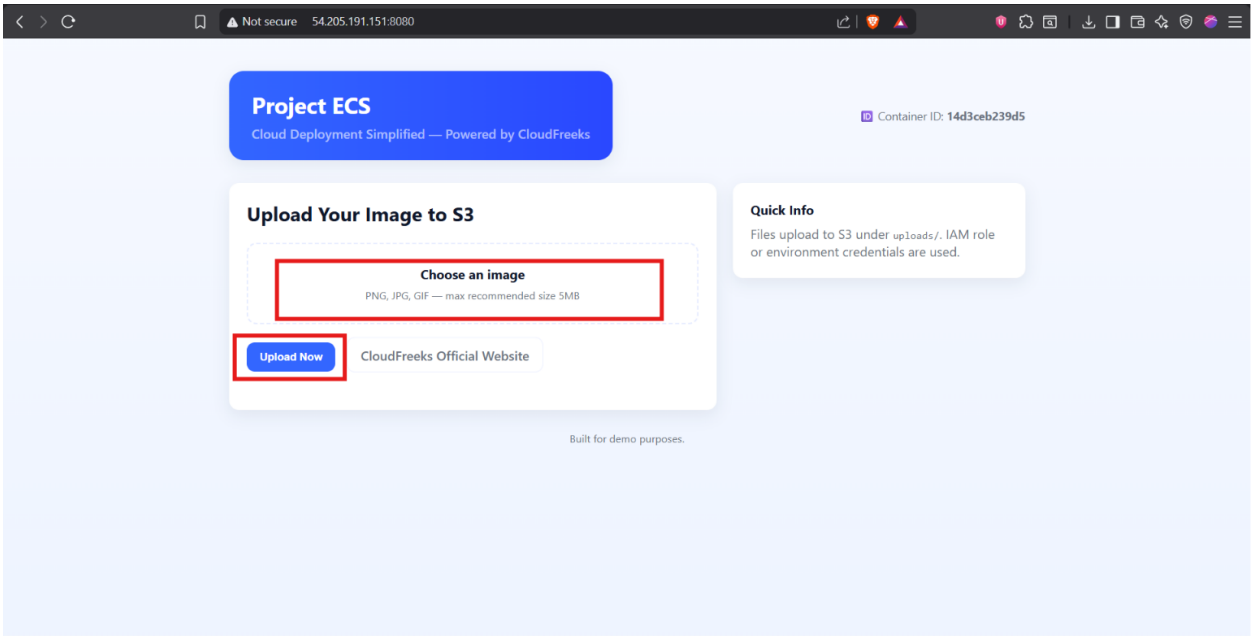
```
http://<EC2-Public-IP>:8080
```

- If the application does not open, navigate to **Amazon EC2 → Security Groups** and confirm that port **8080** is allowed in the inbound rules.

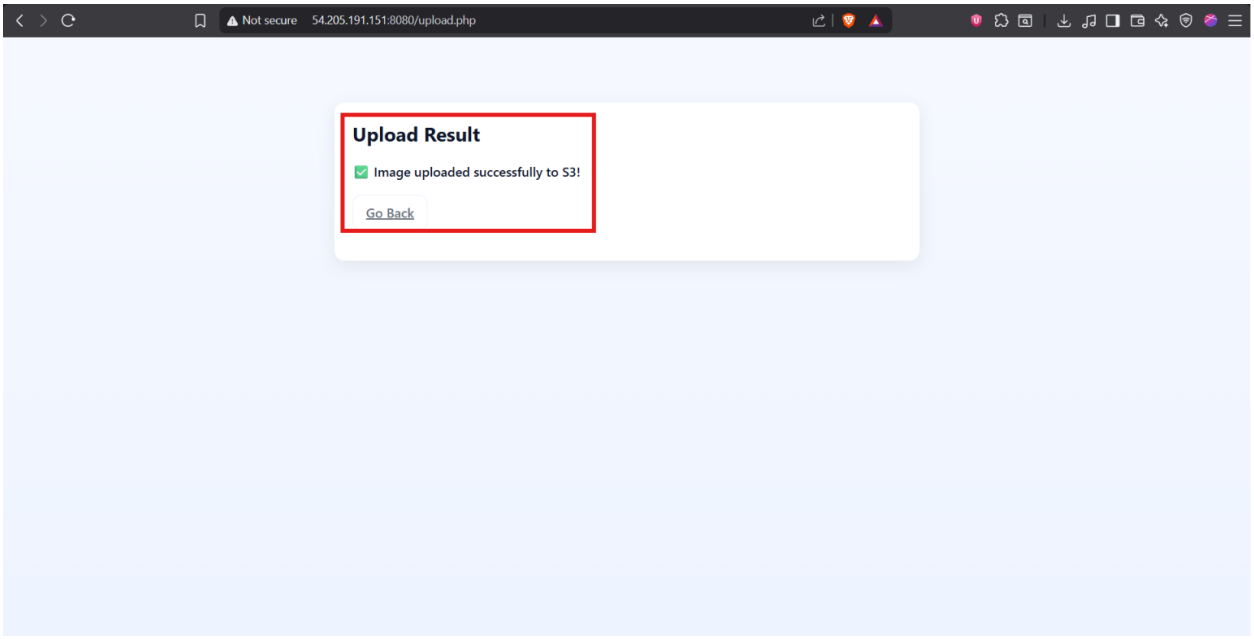


Step 4 – Test the PHP Application

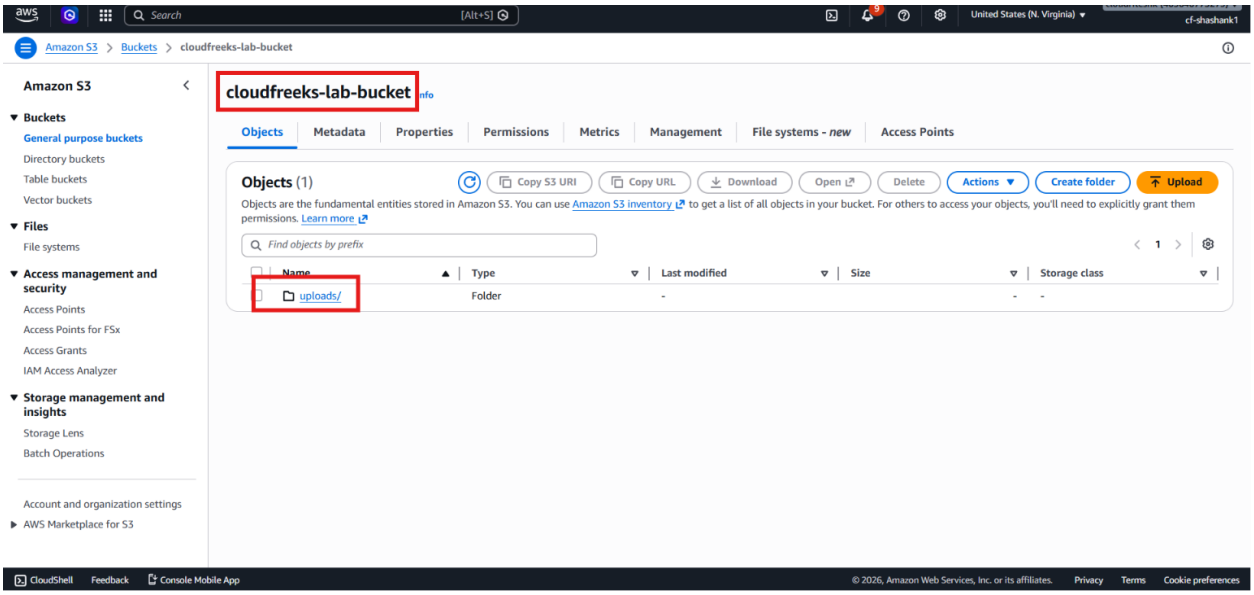
- Upload a sample image using the application.



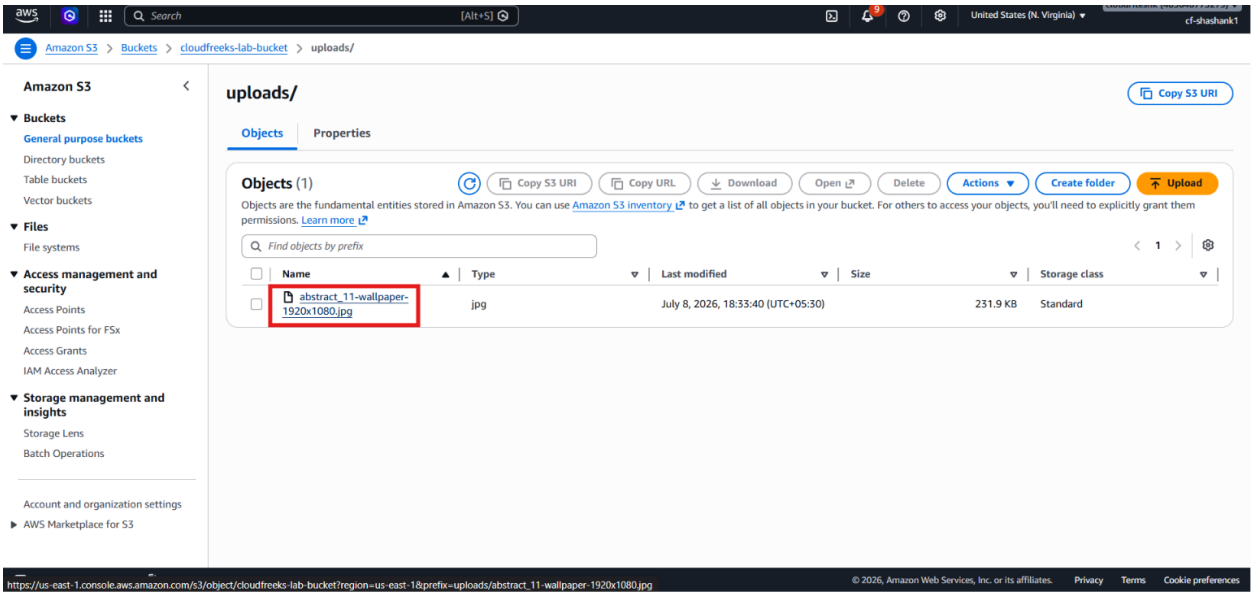
- You get successful upload screen after uploading.



- Navigate to **Amazon S3**.
- Open the configured bucket.
- Confirm that an uploads folder was created automatically.



- Confirm that the uploaded image appears in the bucket and can be opened successfully.



Verification

- Verified that the Docker image was available on the build machine.
- Verified that the Docker container started successfully.
- Verified that the application opened through port **8080**.
- Verified that the image upload succeeded.
- Verified that the uploaded file appeared in the **Amazon S3** bucket.